Pointers, Polymorphism, and Memory Allocation

C++ Interlude 2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Memory Allocation for Variables and Early Binding of Methods

- Declare variable *x* to have data type *int*
  - C++ compiler allocates memory cell to hold an integer
  - Use the identifier *x* to refer to this cell
    - E.g. int x = 5;
- A function's locally declared variables
  - Placed into an activation record with parameters and bookkeeping data
  - Activation record placed on run-time stack
  - Activation record destroyed when function finished

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Memory Allocation for Variables and Early Binding of Methods

- When you create an object, the storage for data members of an object
  - E.g., PlainBox<string> myPlainBox;
  - Placed into an activation record for the currently executing function or method.
  - Data fields placed on the run-time stack just as primitive data types are.
  - if you invoke the setItem method for myPlainBox, compiler knows to invoke the corresponding method for PlainBox
    - myPlainBox.setItem("First item");
- This is early binding, made during compilation
  - Cannot be altered during execution

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Memory Allocation for Variables and Early Binding of Methods

- Automatic memory management and early binding sometimes insufficient
  - Need to take advantage of polymorphism.
  - Must access an object outside of the function or method that creates it.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Early versus Late Binding

```
// C++ Program to illustrate early binding
#include<iostream>
using namespace std;

class Base
{
public:
   void show()
      {cout << " In Base \n"; }
};

class Derived : public Base
{
public:
   void show()
      {cout << "In Derived \n"; }
};

int main()
{
   Base* bp = new Derived;
   bp->show(); // RUN-TIME POLYMORPHISM
   delete bp;
   bp = nullptr;
   return 0;
}
```

This will execute

```
// C++ Program to illustrate late binding
#include<iostream>
using namespace std;

class Base
{
public:
   virtual void show()
      {cout << " In Base \n"; }
};

class Derived : public Base
{
public:
   void show()
      {cout << "In Derived \n"; }
};

int main()
{
   Base* bp = new Derived;
   bp->show(); // RUN-TIME POLYMORPHISM
   delete bp;
   bp = nullptr;
   return 0;
}
```

This will execute

https://www.geeksforgeeks.org/early-binding-late-binding-c/

Early versus Late Binding

- Use early binding for better performance and when you don't need polymorphism.
- Use late binding (virtual functions) for dynamic behavior and when you need to work with objects of different derived classes through a common base class pointer or reference.

https://www.geeksforgeeks.org/early-binding-late-binding-c/

Pointers and Program's Free Store

When a C++ program begins execution, the operating system sets aside memory for

- Run-time stack,
- Code storage – store your code, or text storage,
- Static storage – store any global variables and static variables,
- Heap, or free store, which a programmer can use to store data

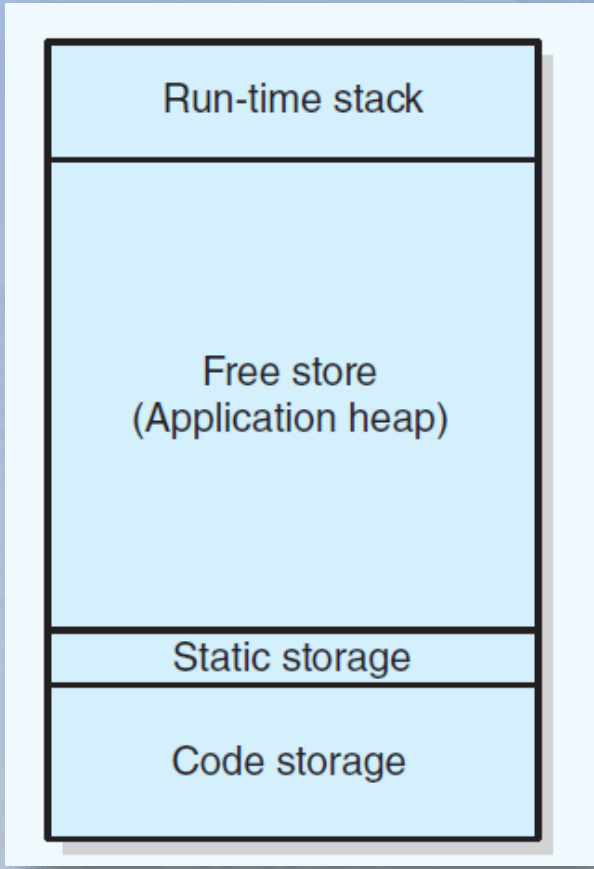
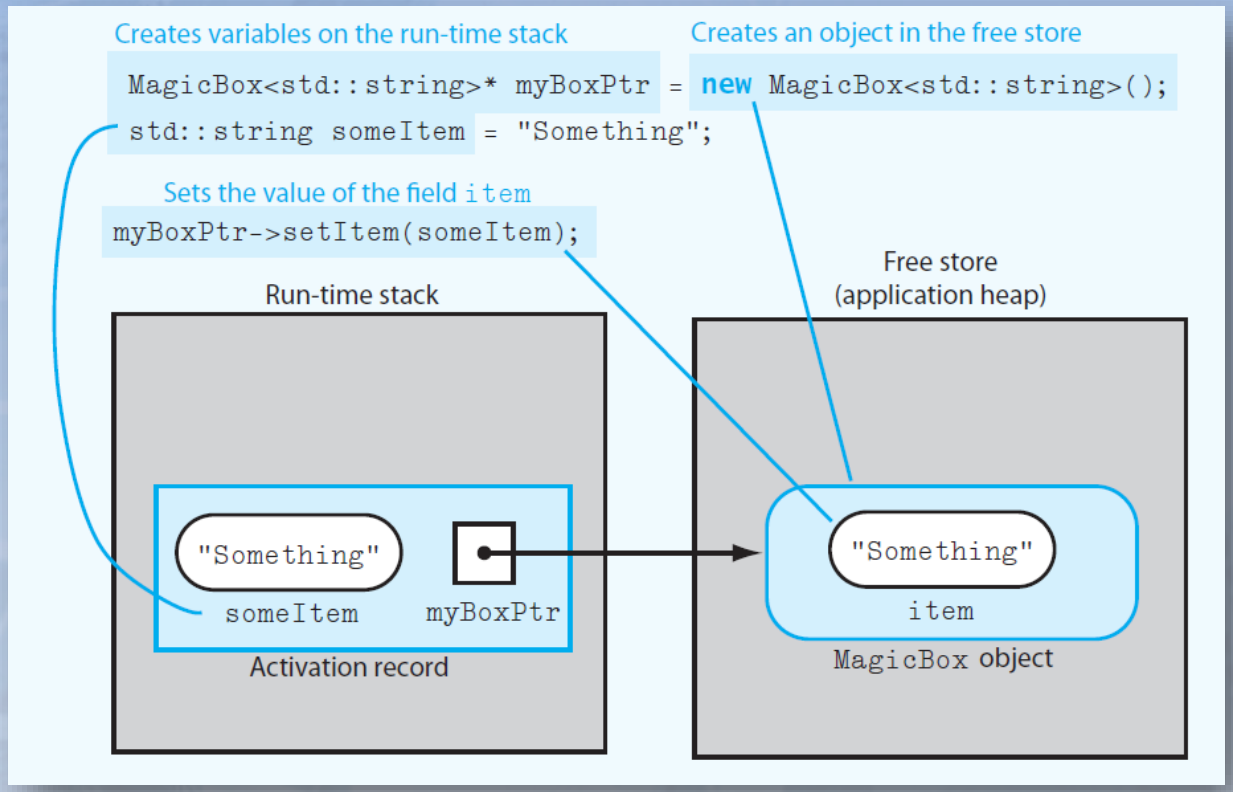


FIGURE C2-2 Sample program memory layout

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Pointers and Program's Free Store



- Variables placed in the heap persist in memory even when the function or method that created them ends
- When a programmer allocates memory from the free store (heap) for a variable, the programmer must deallocate that memory when the variable is no longer needed, else memory leak

FIGURE C2-3 Run-time stack and free store after myboxPtr points to a MagicBox object and its data member item is set

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Pointers and Program's Free Store

This newly created object has no programmer-defined name; the only way to access its methods is indirectly via the pointer that new creates: myBoxPtr->

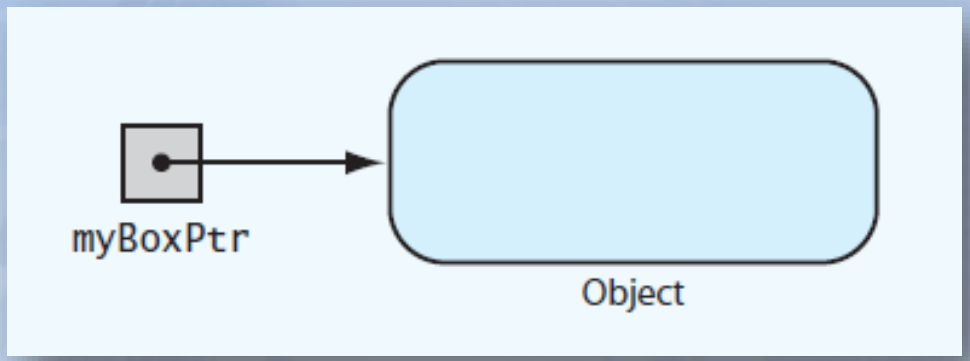


FIGURE C2-4 myBoxPtr and the object to which it points

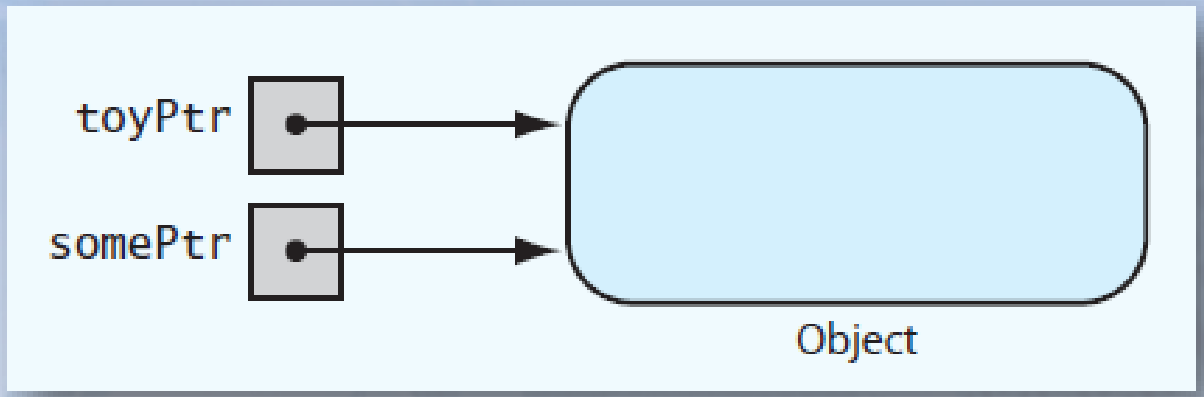
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Pointers and Program's Free Store

If you have two pointer variables that can point to the same type of object, you can make them point to the same object by using the assignment operator:

somePtr = toyPtr; /* somePtr point to same object which toyPtr is pointing to;

note that the object itself is not copied */



If you declare a pointer variable but do not immediately create an object for it to reference, you should set the pointer to nullptr. C++ does not initialize the pointer for you:

```
ToyBox<int>* myToyPtr = nullptr;
```

FIGURE C2-5 Two pointer variables that point to the same object

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Deallocating Memory

- When memory to which pointer variable points is no longer needed
  - Deallocate it by using delete operator.
  - Then set pointer variable to *nullptr*
- Otherwise dangling pointer exists
  - It would still contain address of object that was deallocated.
  - Can be source of serious errors.
- C++ memory leak tools: https://learn.microsoft.com/en-us/cpp/c-runtime-library/find-memory-leaks-using-the-crt-library?view=msvc-170

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

- Memory leaks occur when
  - An object has been created in the heap (free stores), but
  - Program no longer has a way to access

```
void myLeakyFunction(const double& someItem)
{
   ToyBox<double>* someBoxPtr = new ToyBox<double>();
   someBoxPtr->setItem(someItem);
}  // end myLeakyFunction
```

We have memory leak, since someItem and the pointer someBoxPtr are both stored on the stack, which are destroyed after the function ends.

LISTING C2-1 Poorly written function that allocates memory in the free store

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

```
// Creating first object
MagicBox<std::string>* myBoxPtr = new MagicBox<std::string>();

// Creating second object
MagicBox<std::string>* yourBoxPtr = new MagicBox<std::string>();

// Assignment causes an inaccessible object
yourBoxPtr = myBoxPtr;
```

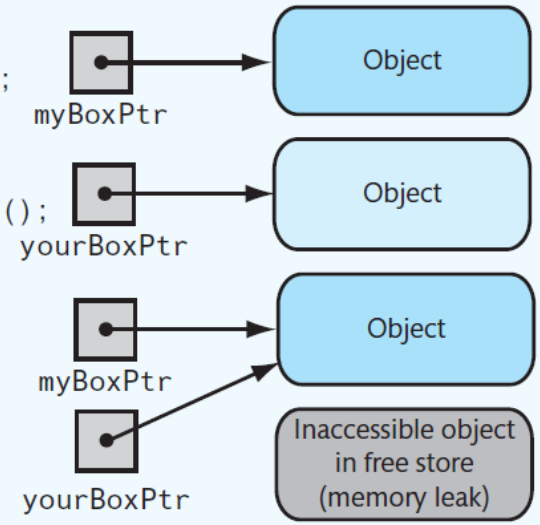


Figure C2-6 An assignment that causes an inaccessible object.

- To prevent memory leak,
  - Do not create the 2nd object
  - Initialize yourBoxPtr to nullptr, or Set it to myBoxPtr
- More subtle memory leak occurs when function or method creates an object in heap, and loses the pointer by not returning that pointer or not storing it in a class member

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

- Memory leak prevention
  - The best option for preventing a memory leak is to not use a function to return a pointer to a newly created object
  - You should define a class that has a method for this task
  - Class takes responsibility for deleting the object in the heap (free store) and ensures that there is no memory leak. At a minimum, such a class will have three parts:
    1. a method that creates the object in the free store,
    2. a data field that points to the object, and
    3. a method—the destructor —that deletes the object when the class instance is no longer needed

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

- Destructors
  - Every C++ class has a destructor that has the same name as the class, but is preceded by the tilde (~) character.
  - For objects that are local variables, the destructor is called when the activation record containing the object is removed from the run-time stack.
  - If the object was created using new and is stored in the free store, the destructor is called when the client uses the operator delete to free the memory allocated to the object.
  - Often, the compiler-generated destructor is sufficient for a class, but if the class itself creates an object on the free store (heap) by using the new operator, it is a safe and secure programming practice to implement a destructor to ensure that the memory for that object is freed

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Dangling Pointers

- Situations that can cause dangling pointer
  - if you do not set a pointer variable to *nullptr* after using delete
  - If you declare a pointer variable but do not assign it a value

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

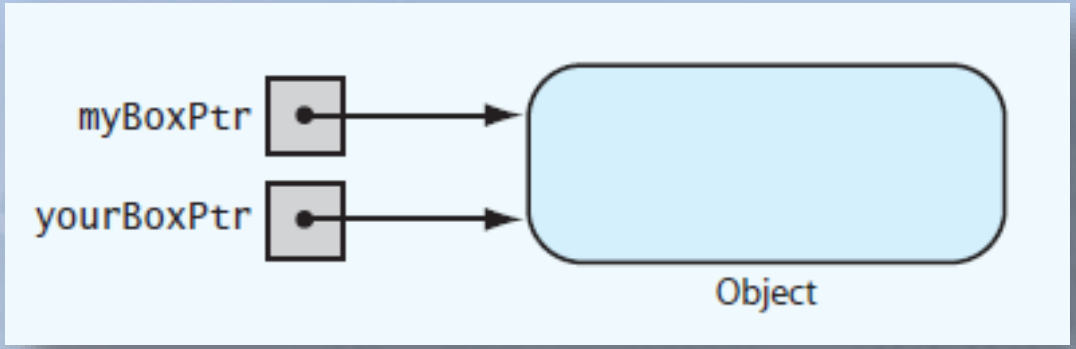
Avoiding Dangling Pointers

If you have two (2) or more pointers pointing to the same object (alias), and

- you delete the first pointer to the object, and
- attempt to execute the method with the second pointer, will result in a runtime error, since the second pointer is a dangling pointer

o MagicBox<string>* myBoxPtr = new MagicBox<string>();

o MagicBox<string>* yourBoxPtr = myBoxPtr;



Suppose we then execute:

```
delete myBoxPtr;
myBoxPtr = nullptr ;
yourBoxPtr->getItem();
```

- Program aborts because yourBoxPtr still references the object's location in the heap, even though the object no longer exists
- yourBoxPtr is a dangling pointer and the program aborts when we try to call a method on that object

FIGURE C2-7 Two pointers referencing (pointing to) the same object

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Dangling Pointers

- Set pointer variables to nullptr either initially or when you no longer need them.
- If a class has a pointer variable as a data field, the constructor should always initialize that data field, to point either to an object or to nullptr.
- Test whether a pointer variable contains nullptr before using it to call a method.
- Try to reduce the use of aliases in your program.
- Do not delete an object in the free store (heap) until you are certain that no other alias needs to use it.
- Set all aliases that reference a deleted object to nullptr when the object is deleted.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods and Polymorphism

- Allow compiler to perform the late binding necessary for polymorphism
  - Declare methods in base class as *virtual*, to enable late binding for polymorphism; indicates methods can be overridden
  - Create variables in the free store (heap), and use pointers to reference them
  - Change function to accept that pointer
  - Free the memory in heap; delete and assign to nullptr

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods and Polymorphism

- Key points about virtual methods
  - A virtual method is one that a derived class can override.
  - A derived class does not need to override an existing implementation of an inherited virtual method.
  - Any of a class's methods may be virtual. However, if you do not want a derived class to override a particular method, the method should not be virtual.
  - Constructors cannot be virtual.
  - Destructors can and should be virtual. Virtual destructors ensure that future descendants of the object can deallocate themselves correctly.
  - A virtual method's return type cannot be overridden

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Dynamic Allocation of Arrays

- An ordinary C++ array is statically allocated

```
const int MAX_SIZE = 50;
double myArray[MAX_SIZE];
```

MAX_SIZE cannot change during program execution

- Can use *new* operator to allocate an array dynamically

```
int arraySize = 50;
double* anArray = new double[arraySize];
```

arraySize can change during program execution

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Dynamic Allocation of Arrays

When you allocate an array dynamically, you need to return its memory cells to the system when you no longer need them:

- delete returns a dynamically allocated array to system for reuse

```
delete [ ] anArray;
```

- Increase size of dynamically allocated array

```
double* oldArray = anArray;                          // Copy pointer to array
anArray = new double[2 * arraySize];                 // Double array size

for (int index = 0; index < arraySize; index++)      // Copy old array
   anArray[index] = oldArray[index];

delete [ ] oldArray;                                 // Deallocate old array
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Interlude 2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Interlude 2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

- Need to write a function – takes two arguments:
  - An object of any of the three types of boxes (from Interlude 1)
  - An item of type *string*
- Function should place item in box by invoking box's *setItem* method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

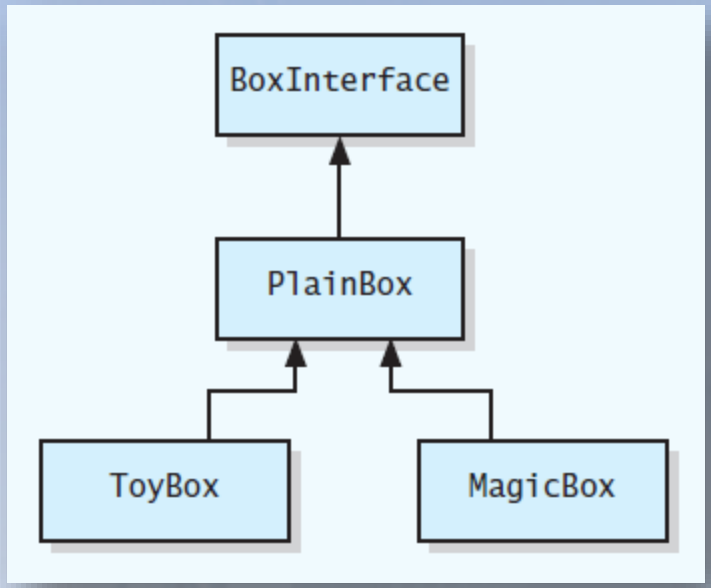


FIGURE C2-1 UML class diagram for a family of classes

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

- You may think this function would suffice

```
void placeInBox(PlainBox<string>& theBox, string theItem)
{
   theBox.setItem(theItem);
}  // end placeInBox
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

- Used in this context

```
std::string specialItem = "Riches beyond compare!";
std::string hammerItem = "Hammer";

PlainBox<std::string> myPlainBox;
placeInBox(myPlainBox, hammerItem);              // Stores hammerItem
placeInBox(myPlainBox, specialItem);             // Stores specialItem
std::cout << myPlainBox.getItem() << std::endl;  // Displays specialItem

MagicBox<std::string> myMagicBox;
placeInBox(myMagicBox, hammerItem);              // Stores hammerItem
placeInBox(myMagicBox, specialItem);             // Stores specialItem
std::cout << myMagicBox.getItem() << std::endl;  // Displays specialItem
```

- Code compiles, but does not perform as you would expect due to

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

- Version of setItem called is determined when the program is compiled.

```
void placeInBox(PlainBox<string>& theBox, string theItem)
{
   theBox.setItem(theItem);
}  // end placeInBox
```

- Need a way to communicate to compiler
  - Code to execute should not be determined until program is running.
  - Called late binding

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

```
/** @file GoodMemory.h */
#ifndef GOOD_MEMORY_
#define GOOD_MEMORY_
#include "ToyBox.h"

class GoodMemory
{
private:
   ToyBox<double>* someBoxPtr;
public:
   GoodMemory();            // Default constructor
   virtual ~GoodMemory(); // Destructor
   void fixedLeak(const double& someItem);
}; // end GoodMemory
#endif
```

LISTING C2-2 Header file for the class GoodMemory

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Memory Leaks

```cpp
/** @file GoodMemory.cpp */
#include "GoodMemory.h"

GoodMemory::GoodMemory() : someBoxPtr(nullptr)
{
}  // end default constructor

GoodMemory::~GoodMemory()
{
   delete someBoxPtr;
}  // end destructor

void GoodMemory::unleakyMethod(const double& someItem)
{
   someBoxPtr = new ToyBox<double>();
   someBoxPtr->setItem(someItem);
}  // end unleakyMethod
```

LISTING C2-3 Implementation file for the class GoodMemory

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Avoiding Dangling Pointers

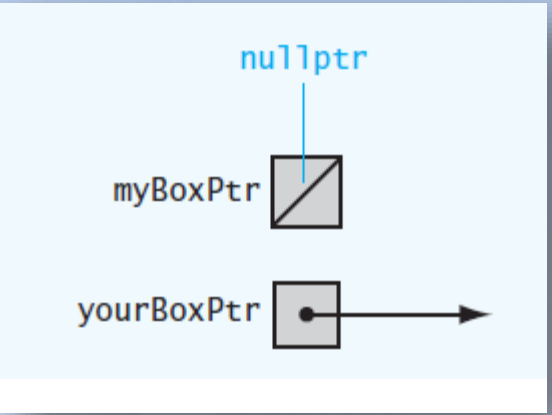


FIGURE C2-8 Example of a dangling pointer

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods and Polymorphism

- Allow compiler to perform the late binding necessary for polymorphism
  - Declare methods in base class as *virtual*.

```
/** @file PlainBox.h */
#ifndef PLAIN_BOX_
#define PLAIN_BOX_

template<class ItemType> // Indicates this is a template

// Declaration for the class PlainBox
class PlainBox
{
private:
   // Data field
   ItemType item;
```

LISTING C2-4 Revised header file for the class PlainBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods and Polymorphism

```
public:
   // Default constructor
   PlainBox();

   // Parameterized constructor
   PlainBox(const ItemType& theItem);

   // Mutator method that can change the value of the data field
   virtual void setItem(const ItemType& theItem);

   // Accessor method to get the value of the data field
   virtual ItemType getItem() const;
}; // end PlainBox

#include "PlainBox.cpp" // Include the implementation file
#endif
```

LISTING C2-4 Revised header file for the class PlainBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods and Polymorphism

```
    // Data field
    ItemType item;

public:
    // Default constructor
    PlainBox();

    // Parameterized constructor
    PlainBox(const ItemType& theItem);

    // Mutator method that can change the value of the data field
    virtual void setItem(const ItemType& theItem);

    // Accessor method to get the value of the data field
    virtual ItemType getItem() const;
}; // end PlainBox

#include "PlainBox.cpp" // Include the implementation file
#endif
```

LISTING C2-4 Revised header file for the class *PlainBox*

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Virtual Methods and Polymorphism

```
std::string specialItem = "Riches beyond compare!";
std::string hammerItem = "Hammer";

PlainBox<std::string>* myPlainBoxPtr = new PlainBox<std::string>();
placeInBox(myPlainBoxPtr, hammerItem);              // Stores hammerItem
placeInBox(myPlainBoxPtr, specialItem);             // Stores specialItem
std::cout << myPlainBoxPtr->getItem() << std::endl; // Displays specialItem

MagicBox<std::string>* myMagicBoxPtr = new MagicBox<std::string>();
placeInBox(myMagicBoxPtr, hammerItem);              // Stores hammerItem
placeInBox(myMagicBoxPtr, specialItem);             // Ignores specialItem
std::cout << myMagicBoxPtr->getItem() << std::endl; // Displays hammerItem
```

To fully implement late binding, create variables in free store and use pointers to reference them

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Resizable Array-Based Bag

- Use a resizable array to implement ADT bag so that bag never becomes full

```
template<class ItemType>
bool ArrayBag<ItemType>::add(const ItemType& newEntry)
{
   bool hasRoomToAdd = (itemCount < maxItems);
   if (!hasRoomToAdd)
   {
      ItemType* oldArray = items;
      items = new ItemType[2 * maxItems];
      for (int index = 0; index < maxItems; index++)
         items[index] = oldArray[index];
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Resizable Array-Based Bag

- Use a resizable array to implement ADT bag so that bag never becomes full

```
        for (int index = 0; index < maxItems; index++)
            items[index] = oldArray[index];
        delete [ ] oldArray;
        maxItems = 2 * maxItems;
    }  // end if

    // We can always add the item
    items[itemCount] = newEntry;
    itemCount++;
    return true;
}  // end ResizableArrayBag add
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved